

It has been suggested that *LEB128* be merged into this article. (Discuss)

[Learn more](#)

A **variable-length quantity** (VLQ) is a [universal code](#) that uses an arbitrary number of [binary octets](#) (eight-bit bytes) to represent an arbitrarily large [integer](#). A VLQ is essentially a base-128 representation of an unsigned integer with the addition of the eighth bit to mark continuation of bytes. See the example below.

Applications and history

Base-128 compression is known by many names – VB (Variable Byte), **VByte**, **Varint**, **VInt**, **EncInt** etc.^[1]

A **variable-length quantity** (VLQ)  was defined for use in the standard [MIDI file](#) format^[2] to save additional space for a resource constrained system, and is also used in the later [Extensible Music Format \(XMF\)](#).

Base-128 is also used in [ASN.1](#) BER encoding to encode tag numbers and [Object Identifiers](#).^[3] It is also used in the [WAP](#) environment, where it is called **variable length unsigned integer** or **uintvar**. The [DWARF](#) debugging format^[4] defines a variant called [LEB128](#) (or **ULEB128** for unsigned numbers), where the least significant group of 7 bits are encoded in the first byte and the most significant bits are in the last byte (so effectively it is the little-endian analog of a variable-length quantity). [Google Protocol Buffers](#) use a similar format to have compact representation of integer values,^[5] as does [Oracle](#) Portable Object Format (POF)^[6] and the [Microsoft .NET Framework](#) "7-bit encoded int" in the **BinaryReader** and **BinaryWriter** classes.^[7]

It is also used extensively in web browsers for [source mapping](#) – which contain a lot of integer line & column number mappings – to keep the size of the map to a minimum.^[8]

Variable width integers in [LLVM](#) use a similar principle. The encoding chunks are little-endian and need not be 8 bits in size. The LLVM documentation describes a field that uses 4-bit chunk, with each chunk consisting of 1 bit continuation and 3 bits payload.^[9]

General structure

The encoding assumes an octet (an eight-bit byte) where the most significant bit (MSB), also commonly known as the [sign bit](#), is reserved to indicate whether another VLQ octet follows.

VLQ Octet

7	6	5	4	3	2	1	0
2^7	2^6	2^5	2^4	2^3	2^2	2^1	2^0
A	B_n						

If A is 0, then this is the last VLQ octet of the integer. If A is 1, then another VLQ octet follows.

B is a 7-bit number [0x00, 0x7F] and n is the position of the VLQ octet where B_0 is [the least significant](#). The VLQ octets are arranged [most significant first](#) in a stream.

Variants

The general VLQ encoding is simple, but in basic form is only defined for [unsigned integers](#) (nonnegative, positive or zero), and is somewhat redundant, since prepending 0x80 octets corresponds to zero padding. There are various [signed number representations](#) to handle negative numbers, and techniques to remove the redundancy.

The **GroupVB** (Group Varint Encoding) factors out the continuation flags for four consecutive values into a single header byte followed by a variable number of data bytes. The GroupVB layout [reduces CPU branches](#), making GroupVB faster than basic base-128 on deeply pipelined CPUs.^[1]

Signed numbers

Sign bit

Negative numbers can be handled using a [sign bit](#), which only needs to be present in the first octet.

In the data format for Unreal Packages used by the [Unreal Engine](#), a variable length quantity scheme called Compact Indices^[10] is used. The only difference in this encoding is that the first VLQ has the sixth bit reserved to indicate whether the encoded integer is positive or negative. Any consecutive VLQ octet follows the general structure.

Unreal Signed VLQ

First VLQ Octet								Other VLQ Octets							
7	6	5	4	3	2	1	0	7	6	5	4	3	2	1	0
2^7	2^6	2^5	2^4	2^3	2^2	2^1	2^0	2^7	2^6	2^5	2^4	2^3	2^2	2^1	2^0
A	B	C_0						A	$C_n (n>0)$						

If A is 0, then this is the last VLQ octet of the integer. If A is 1, then another VLQ octet follows.

If B is 0, then the VLQ represents a positive integer. If B is 1, then the VLQ represents a negative number.

C is number chunk being encoded and n is the position of the VLQ octet where C_0 is [the least significant](#). The VLQ octets are arranged [most significant first](#) in a stream.

Zigzag encoding

An alternative way to encode negative numbers is to use the least-significant bit for sign. This is notably done for Google Protocol Buffers, and is known as a **zigzag encoding** for [signed integers](#).^[11] One can encode the numbers so that encoded 0 corresponds to 0, 1 to −1, 10 to 1, 11 to −2, 100 to 2, etc.: counting up alternates between nonnegative (starting at 0) and negative (since each step changes the least-significant bit, hence the sign), whence the name "zigzag encoding". Concretely, transform the integer as $(n \ll 1) \oplus (n \gg k - 1)$ for fixed k -bit integers.

Two's complement

LEB128 uses [two's complement](#) to represent signed numbers. In this scheme of representation, n bits encodes a range from -2^{n-1} to $2^{n-1}-1$, and all negative numbers start with a 1 in the most significant bit. In Signed LEB128, the input is [sign extended](#) so that its length is a multiple of 7 bits. From there the encoding proceeds as usual.^[12]

In LEB128, the stream is arranged least significant first.^[12]

Removing redundancy

With the VLQ encoding described above, any number that can be encoded with N octets can also be encoded with more than N octets simply by prepending additional 0x80 octets as zero-padding. For example, the decimal number 358 can be encoded as the 2-octet VLQ 0x8266 or

the number 0358 can be encoded as 3-octet VLQ 0x808266 or 00358 as the 4-octet VLQ 0x80808266 and so forth.

However, the VLQ format used in [Git^{\[13\]}](#) removes this prepending redundancy and extends the representable range of shorter VLQs by adding an offset to VLQs of 2 or more octets in such a way that the lowest possible value for such an (N+1)-octet VLQ becomes exactly one more than the maximum possible value for an N-octet VLQ. In particular, since a 1-octet VLQ can store a maximum value of 127, the minimum 2-octet VLQ (0x8000) is assigned the value 128 instead of 0. Conversely, the maximum value of such a 2-octet VLQ (0xff7f) is 16511 instead of just 16383. Similarly, the minimum 3-octet VLQ (0x808000) has a value of 16512 instead of zero, which means that the maximum 3-octet VLQ (0xffff7f) is 2113663 instead of just 2097151. And so forth.

In this way, there is one and only one encoding of each integer, making this a base-128 [bijective numeration](#).

Examples

Diagram showing how to convert 106,903 from decimal to uintvar representation

Here is a worked out example for the decimal number [137](#):

- Represent the value in binary notation (e.g. 137 as 10001001)
- Break it up in groups of 7 bits starting from the lowest significant bit (e.g. 137 as 0000001 0001001). This is equivalent to representing the number in [base 128](#).
- Take the lowest 7 bits and that gives you the least significant byte (**0000 1001**). This byte comes last.
- For all the other groups of 7 bits (in the example, this is 000 0001), set the [MSB](#) to 1 (which gives **1000 0001** in our example). Thus 137 becomes **1000 0001 0000 1001** where the bits in boldface are something we added. These added bits denote if there is another byte to follow or not. Thus, by definition, the very last byte of a variable length integer will have 0 as its [MSB](#).

Another way to look at this is to represent the value in base-128, and then set the MSB of all but the last base-128 digit to 1.

The Standard MIDI File format specification gives more examples:^{[\[2\]](#)[\[14\]](#)}

Integer (decimal)	Integer (hexadecimal)	Integer (binary)	Variable-length quantity (hexadecimal)	Variable-length quantity (binary)
0	0x00000000	00000000 00000000 00000000 00000000	0x00	00000000
127	0x0000007F	00000000 00000000 00000000 01111111	0x7F	01111111
128	0x00000080	00000000 00000000 00000000 10000000	0x81 0x00	10000001 00000000
8192	0x00002000	00000000 00000000 00100000 00000000	0xC0 0x00	11000000 00000000
16383	0x00003FFF	00000000 00000000 00111111 11111111	0xFF 0x7F	11111111 01111111
16384	0x00004000	00000000 00000000 01000000 00000000	0x81 0x80 0x00	10000001 10000000 00000000
2097151	0x001FFFFFFF	00000000 00011111	0xFF 0xFF 0x7F	11111111

		11111111 11111111		11111111 01111111
2097152	0x00200000	00000000 00100000 00000000 00000000	0x81 0x80 0x80 0x00	10000001 10000000 10000000 00000000
134217728	0x08000000	00001000 00000000 00000000 00000000	0xC0 0x80 0x80 0x00	11000000 10000000 10000000 00000000
268435455	0x0FFFFFFF	00001111 11111111 11111111 11111111	0xFF 0xFF 0xFF 0x7F	11111111 11111111 11111111 01111111

References

1. Jianguo Wang; Chunbin Lin; Yannis Papakonstantinou; Steven Swanson. "An Experimental Study of Bitmap Compression vs. Inverted List Compression" . 2017.
doi:10.1145/3035918.3064007
2. MIDI File Format: Variable Quantities
3. <http://www.itu.int/ITU-T/studygroups/com17/languages/X.690-0207.pdf>
4. DWARF Standard
5. Google Protocol Buffers
6. Oracle Portable Object Format (POF)
7. [System.IO.BinaryWriter.Write7BitEncodedInt\(int\) method](#) and [System.IO.BinaryReader.Read7BitEncodedInt\(\) method](#)
8. Introduction to javascript source maps
9. "LLVM Bitcode File Format", section "Variable Width Integers" . Accessed 2019-10-01.
10. <http://unreal.epicgames.com/Packages.htm>

11. Protocol Buffers: Encoding: [Signed Integers](#)
12. Free Standards Group (December 2005). "[DWARF Debugging Information Format Specification Version 3.0](#)" (PDF). p. 70. Retrieved 2009-07-19.
13. <https://github.com/git/git/blob/7fb6aefd2aaffe66e614f7f7b83e5b7ab16d4806/varint.c#L4>
14. [Standard MIDI-File Format Spec. 1.1 \(PDF\)](#)

External links

- [MIDI Manufacturers Association](#) (MMA) - Source for English-language MIDI specs
 - [Association of Musical Electronics Industry](#) (AMEI) -Source for Japanese-language MIDI specs
-
-